



자연어 처리와 컴퓨터비전 심층학습

10장: 비전 트랜스포머

⚙ Status	Not started
👤 Assignee	 시현 이
📅 Due date	05/04/2026
🔖 Task type	개념및코드필사
≡ Description	자연어 처리와 컴퓨터비전 심층학습 10장: 비전 트랜스포머

Intro

ViT(Vision Transformer)

: 구글의 논문을 통해 소개된 이미지 인식을 위한 딥러닝 모델-이미지를 자연어 처리 방식처럼 분류해보려는 시도

- CNN 모델의 합성곱 계층 방법을 사용하지 않고 트랜스포머 모델에서 사용되는 self attention을 적용(이미지를 한번에 처리)
- 이미지 패치가 왼→오, 위→아래 방향으로 순차적으로 입력돼 2차원의 구조의 이미지 특성을 온전히 반영X
 - 이 문제를 해결하기 위해 물체의 크기나 해상도를 계층적으로 학습하는 '**스윈 트랜스포머(Swin Transformer)**'와 '**CvT(Convolutional Vision Transformer)**'모델이 제안됨.
 - 스윈 트랜스포머
 - **로컬 윈도우(Local Window)**를 활용해 각 계층의 어텐션이 되는 패치의 크기와 개수를 다양하게 구성해 이미지의 특징을 학습시킴

- 기존 셀프 어텐션을 로컬 윈도우 안에 대한 어텐션, 로컬 윈도우 간의 어텐션으로 수행해 이미지 특징을 계층적으로 학습시킴
 - 이 어텐션 함수에는 상대적 위치 편향을 반영해 어텐션 값 자체에 위치적 정보 포함
- CvT
 - 기존 합성곱 연산 과정을 ViT에 적용한 모델로, **저수준 특징(Low-level Feature)**과 **고수준 특징(High-level Feature)**을 계층적으로 반영할 수 있음
 - ex) 사람의 저수준 특징-눈,코,입//고수준 특징-눈,코,입 포함한 전체 얼굴
 - 어텐션 연산 과정에서 Query(Q), Key(K), Value(V) 중 키와 값을 기존 특징 벡터보다 축소해 계산 복잡도를 감소시킴

ViT



BERT VS ViT

- 트랜스포머 모델의 구조 그대로 사용하지만, 입력 데이터를 만드는 과정이 서로 다름
- BERT는 작은 단위의 토큰들이 순차적으로 입력됨 ↔ ViT 모델은 이미지가 격자로 작은 단위의 이미지 패치로 나뉘어 순차적으로 입력됨(sequential array)

합성곱 모델과 ViT 모델 비교

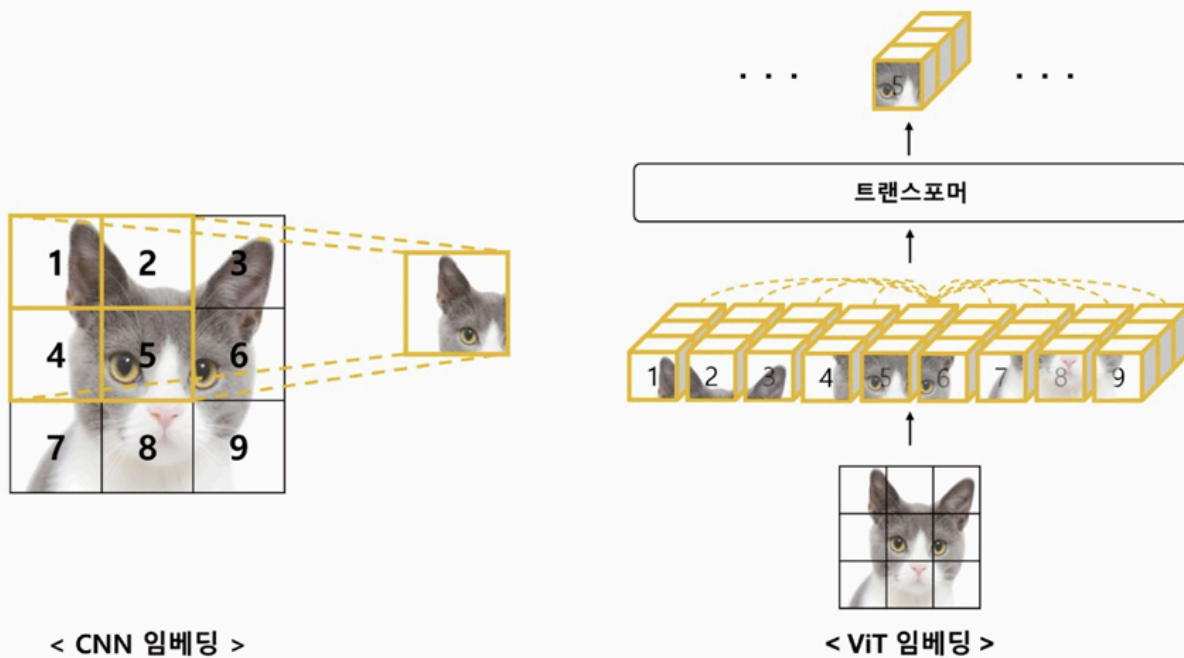


그림 10.3 합성곱 신경망과 ViT 계층의 차이

- **합성곱 신경망 임베딩**
 - 이미지 패치 중 일부만 선택해 학습
 - 좁은 수용 영역(Receptive Field, RF)을 가짐 → 多계층 필요
 - 이미지 크기 맞추는 전처리 필요
 - 이미지의 공간적인 위치 정보를 고려
- **ViT 임베딩**
 - 이미지를 작은 패치들로 나눠 각 패치 간의 상관관계를 학습

- 셀프 어텐션 방법을 사용해 모든 이미지 패치가 서로에게 주는 영향을 고려해 이미지의 전체 특징을 추출함
- Attention Distance를 계산해 오직 한 개의 ViT 레이어로 전체 이미지 정보를 쉽게 표현할 수 있음
- 패치단위 → 효율성
- 패치 간의 상대적인 위치 정보만 고려 → 이미지 변환에 취약

ViT의 귀납적 편향

Inductive Bias(귀납적 편향)

: 일반화 성능 향상을 위한 모델의 가정

: 해당 모델이 가지는 구조와 매개변수들이 데이터에 적합한 가정을 하고 있음을 나타냄

→ 더 적은 데이터로 높은 일반화 성능 보임

→ But, 너무 강하면 다른 유형의 데이터나 관계를 표현하는데 어려움 초래

표 10.1 딥러닝 모델의 귀납적 편향

딥러닝 모델	귀납적 편향
합성곱 신경망(CNN)	지역적 편향
순환 신경망(RNN)	순차적 편향
ViT	귀납적 편향이 거의 없음

- 합성곱 신경망 → 지역적 편향
- 순환 신경망 → 순차적 편향 :
 - 순환 신경망은 이전 시간의 상태를 기억하고 현재 입력과 결합해 다음 상태 예측
 - 시계열 데이터에서 좋은 성능

- but, 특정 관계 편향이 강할수록, 다른 다양한 관계를 표현하기 어려워 귀납적 편향이 약한 모델을 선호하게 됨
- ViT → Q,K,V의 임베딩 형태로 일반화된 관계를 학습 → 귀납적 편향이 거의 X
 - 대용량 데이터에 대해서 이미지 특징들의 관계를 잘 학습

ViT 모델

- 입력 이미지를 transformer 구조에 맞게 일정한 크기의 패치로 나눈 다음 각 패치를 벡터 형태로 변환하는 **패치 임베딩(Patch Embedding)**과 각 패치와의 관계를 학습하는 **인코더(Encoder)** 계층으로 구성됨

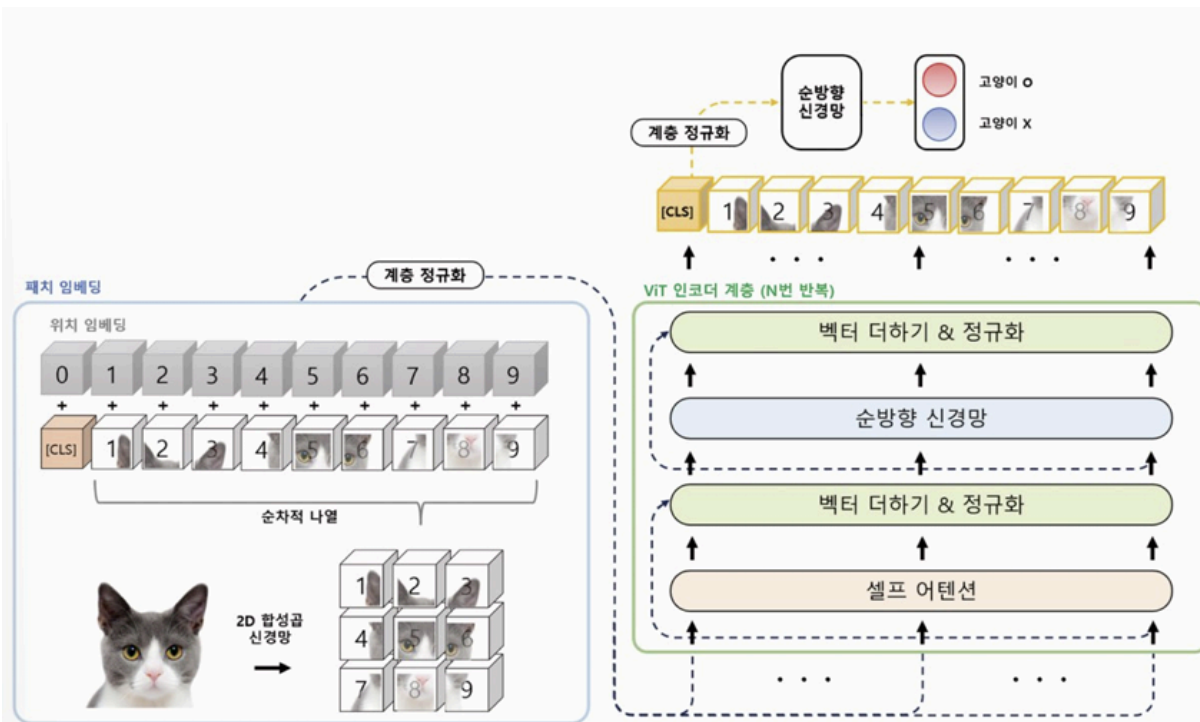


그림 10.4 ViT 모델 구조

패치 임베딩

:입력 이미지를 작은 패치로 분할하는 과정 → 이미지 크기 맞추는 전처리 수행 필요

ex) 640*480 → 224*224

→ 패치 크기로 분할해 시퀀셜 배열 생성

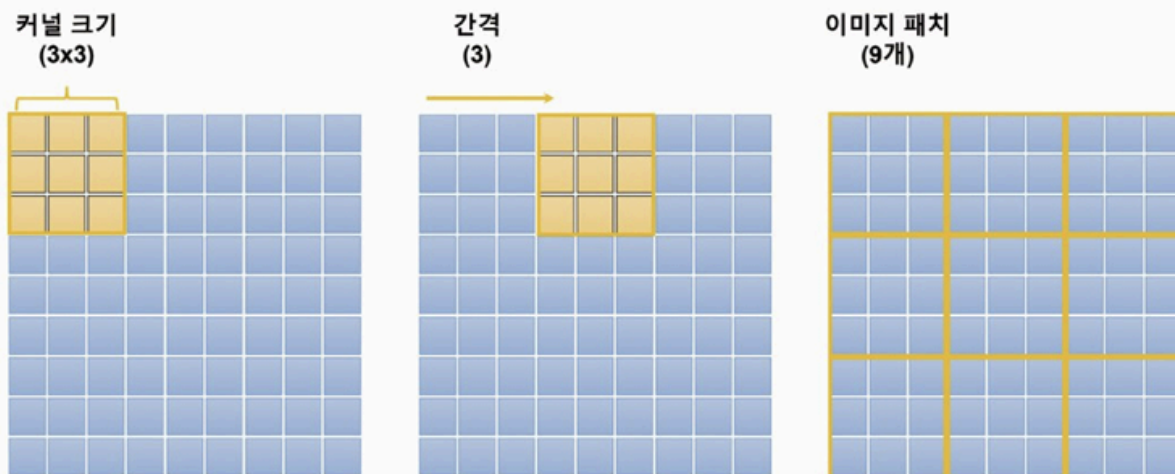


그림 10.5 이미지 패치 생성 과정

- 커널 크기(패치의 크기)와 간격(패치가 이동하는 폭)을 설정(하이퍼파라미터로 정의)
 - ex) 이미지 크기 9*9 ,커널 크기 3*3, 간격이 3 $\Rightarrow$ 9(3*3)개의 이미지 패치가 생성

수식 10.1 패치 계산 과정

$$patch\ size = \frac{image\ size - kernel\ size}{stride} = \frac{9 - 3}{3} + 1 = 3$$

$\Rightarrow$ 가로 패치 3개, 세로 패치도 3개 생성 $\Rightarrow$ 총 9개의 패치가 생성됨

커널 크기가 3*3, 간격이 1이면 $\Rightarrow (9-3+1)*(9-3+1) = 49$ 개의 패치 생성

왼 $\rightarrow$ 오 $\rightarrow$ 위 $\rightarrow$ 아래 순서로 배열

- 배열 가장 왼쪽에 분류 토큰(Special Classification Token, [CLS])을 추가
 - 전체 이미지를 대표하는 벡터로 특정 문제를 예측하는데 사용됨
- 위치 임베딩을 사용해 인접한 패치 간의 관계를 학습
 - 패치의 위치 정보를 임베딩 벡터로 변환하고 기존 이미지 패치 벡터들과 더함
- 계층 정규화를 적용하면 패치 임베딩이 만들어짐

⇒GPU 메모리의 한계를 극복하고 더 큰 이미지 처리할 수 있음

인코더 계층

ViT 모델은 멀티 헤드 어텐션의 학습 과정과 동일

- N개의 인코더 레이어를 반복적으로 적용한 후, 마지막 레이어에서는 분류 토큰이라고 불리는 특별한 패치의 특징 벡터를 추출
 - 분류 토큰 벡터 → 이미지 데이터를 잘 표현하는 특징 벡터로 간주됨
 - ex) 고양이인지 아닌지 구분하는 모델을 학습 : ViT 모델은 시퀀셜 산출물을 모두 사용하는 것이 아니라 전체 이미지의 특징을 잘 표현하는 분류 토큰 벡터만 사용해서 분류 문제를 풀어냄
 - 분류 토큰 벡터를 순방향 신경망에 연결하고 학습

Swin Transformer

: 대규모 비전 인식 모델

- 기존 트랜스포머 기반 모델의 공통적인 문제점
 - Semantic Segmentation 과 같은 작업에 적합하지 X
 - transformer의 self attention은 입력 이미지 크기에 대한 2차 계산 복잡도를 가짐 → 고해상도 이미지를 처리하기 어려움
- 이를 해결하기 위해 hierarchy 특징 맵을 구성해 1차 계산 복잡도를 갖는 swin transformer가 등장
- Shifted Window : 입력 이미지를 일정한 크기의 패치로 분량라고 이 패치들을 쌓아 윈도우를 구성, 구성된 윈도우 영역만 셀프 어텐션을 계산
 - 높은 해상도와 다양한 크기의 객체를 효율적으로 처리

ViT와 스윈 트랜스포머 차이

- ViT 모델

- 획일적인 패치 크기와 위치에 대한 제약 → 다양한 이미지의 크기와 종횡비를 가진 데이터 세트에 대해서는 적용 어려움
- 패치 단위 입력 → 이미지 공간 정보가 완전히 보존되지 않을 수 있음
- 이미지 패치 수 증가 → 계산량 증가 → 컴퓨팅 자원 사용 多
- Swin Transformer
 - Local Window를 활용 → 물체의 크기나 해상도를 계층적으로 학습
 - 그림의 회색 격자==패치, 빨간색 격자==로컬 윈도우
 - 계층적 접근 방식 → 로컬 윈도우가 각 패치의 세부 정보에 집중. 시프트 윈도우를 통해 전역 특징 확인. 고해상도 이미지 효율적으로 처리

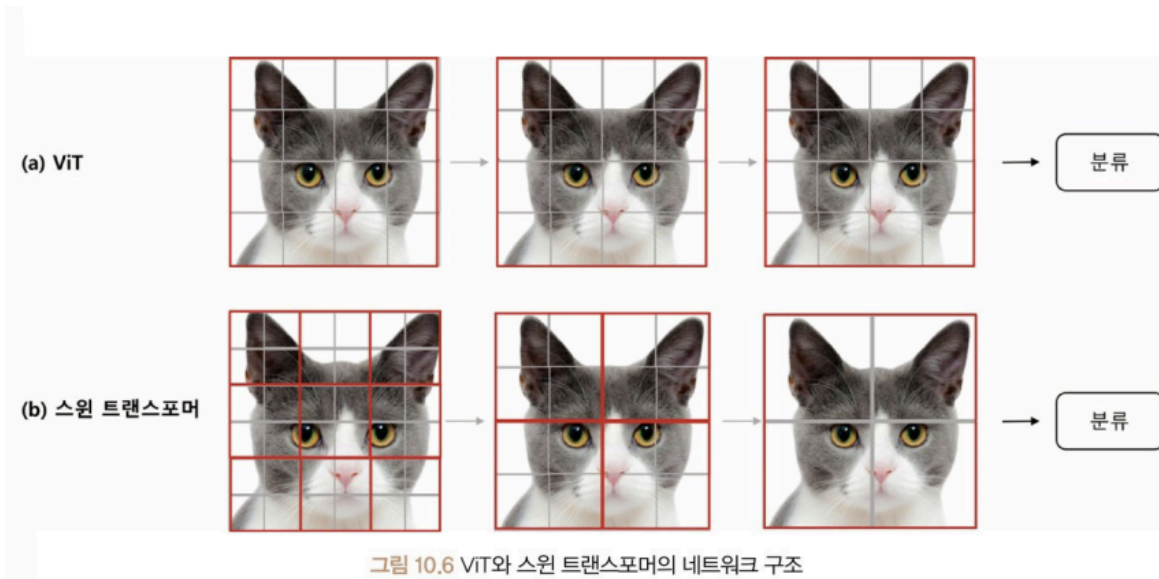


표 10.4 ViT와 스윈 트랜스포머 비교

	ViT	스윈 트랜스포머
분류 헤드	[CLS] 토큰 사용	각 토큰의 평균값 사용
로컬 윈도우 사용	X	O
상대적 위치 편향 사용	X	O
계층별 패치 크기	동일함	다양함

스윈 트랜스포머 모델 구조

: 이미지를 패치 파티션으로 구분 → 선형 임베딩, 스윈 트랜스포머 블록, 패치 병합 연산이 반복적으로 수행됨

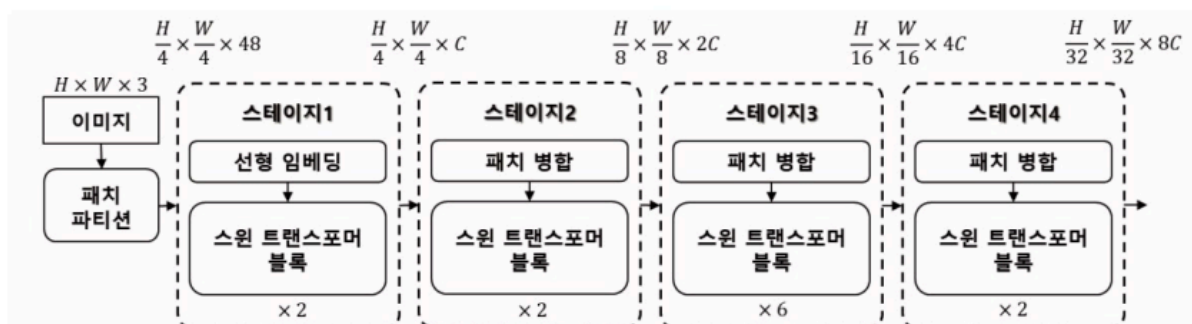


그림 10.7 스윈 트랜스포머 구조

1. 입력 이미지를 일정한 크기의 패치로 분할
2. 각 패치에 대해 선형 임베딩 수행 → 각 패치는 고정된 차원의 벡터로 변환
3. 분할된 패치들을 기반으로 스윈 트랜스포머 블록을 구성
 - a. 어텐션 계층, 계층 정규화, 다층 퍼셉트론 등을 포함해 패치 간 상호작용 수행
4. 패치를 병합해 전체 이미지에 대한 분류 수행
 - a. 분할된 패치들을 순차적으로 합치면서 이미지의 전체적인 정보를 추출하는 방식 사용
 - b. 패치 병합은 모델의 매개변수를 줄이기 위해 이미지 텐서 $[C, H, W]$ 를 $[2C, H/2, W/2]$ 로 재정렬해 저차원 임베딩을 수행

패치 파티션/ 패치 병합

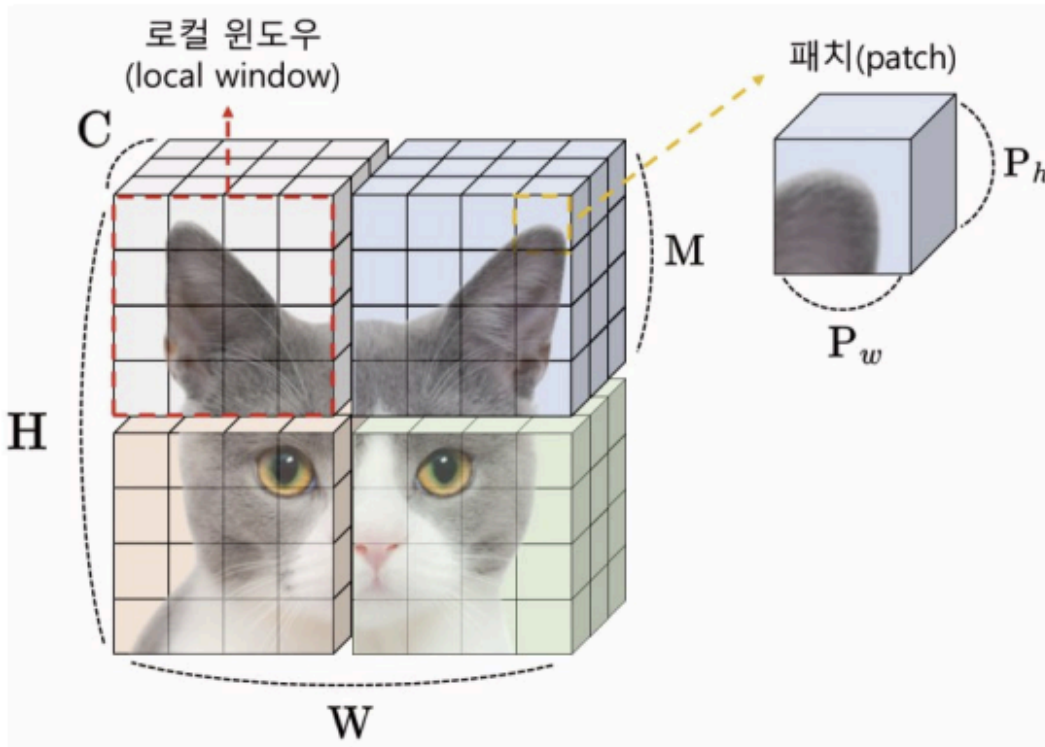


그림 10.8 패치 파티션 구조

- 패치 파티션(Patch Partition)
 - 입력 이미지를 작은 사각형 패치로 분할해 처리하는 방식
 - 분할된 패치는 트랜스포머 계층의 입력으로 사용되며, 입력 이미지의 공간 정보를 보존하고 트랜스포머 계층에 대한 계산 효율성을 높임
 - M은 로컬 윈도우의 크기를 의미
- 패치 병합(Patch Merging)
 - 인접한 패치들의 정보를 저차원으로 축소하는 과정
 - 모델의 매개변수 줄임 → $[C, H, W]$ 를 $[2C, H/2, W/2]$ 로 재정렬한 후, $2C$ 의 차원을 저차원으로 임베딩
 - ex) $[3, 8, 8]$ 이미지 → $[6, 4, 4]$ 로 재정렬 → 증가된 6차원 채널을 원래 3차원으로 임베딩해 산출물 $[3, 4, 4]$ 텐서를 생성

스윈 트랜스포머 블록

: 패치 파티션 이후 네개의 스테이지 안에서 반복 적용되며, 선형 임베딩 또는 패치 병합 이후에 수행됨

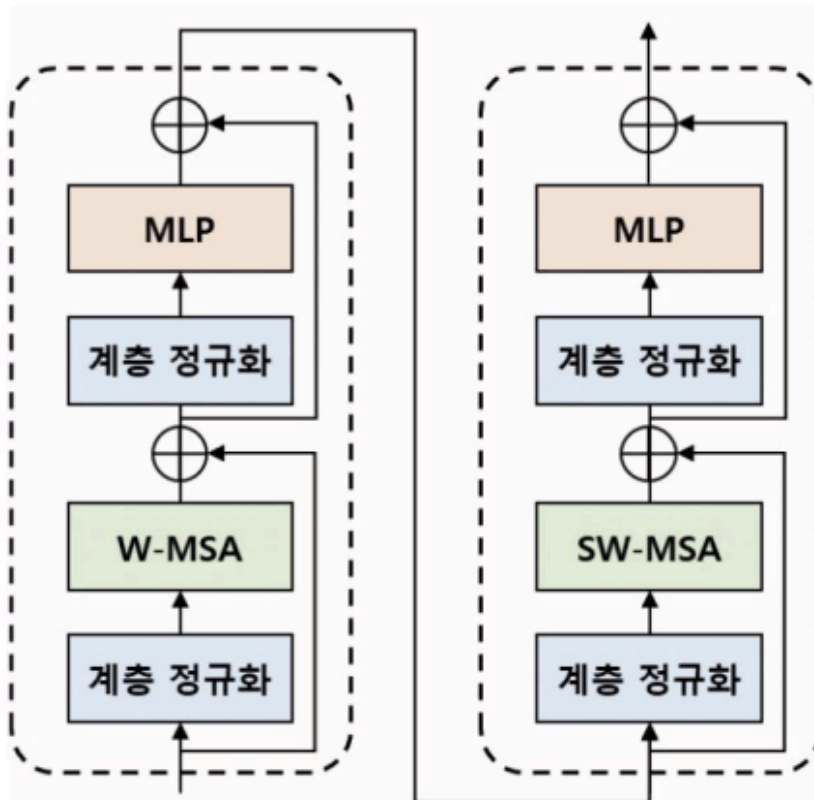


그림 10.9 스윈 트랜스포머 블록 구조

- 계층 정규화, W-MSA(Window Multi-head Self-Attention), MLP, SW-MSA(Shifted Window Multi-head Self-Attention) 등으로 구성됨
 - W-MSA: 로컬 윈도우를 사용하는 멀티 헤드 셀프 어텐션으로, 입력 특징 맵을 중첩되지 않게 나눈 다음 각 윈도우에서 독립적으로 셀프 어텐션을 수행
 - 이미지에서 일정한 크기의 윈도우 영역을 설정, 해당 영역 내 픽셀 간의 어텐션을 계산
 - 이미지 내의 전체적인 어텐션을 수행, 특정 위치에 대한 정보를 추출
 - 각 윈도우 내의 지역적인 특징을 분석

- 전체 입력에 대한 셀프 어텐션 비용을 2차 계산 복잡도에서 1차 계산 복잡도로 줄임(효율적인 배치 계산 통해 연산량 대폭 감소)
- 단점: 윈도우 내부의 이미지 패치 영역에만 셀프 어텐션 수행

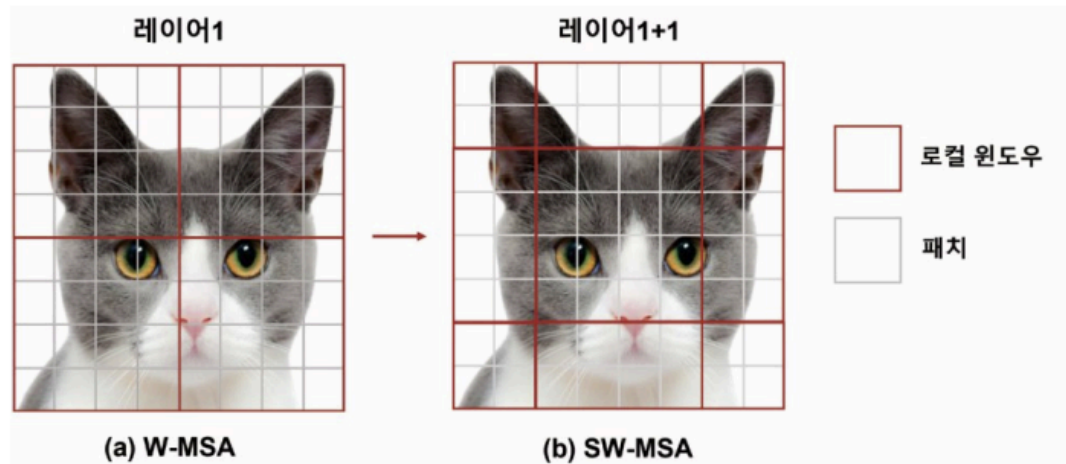
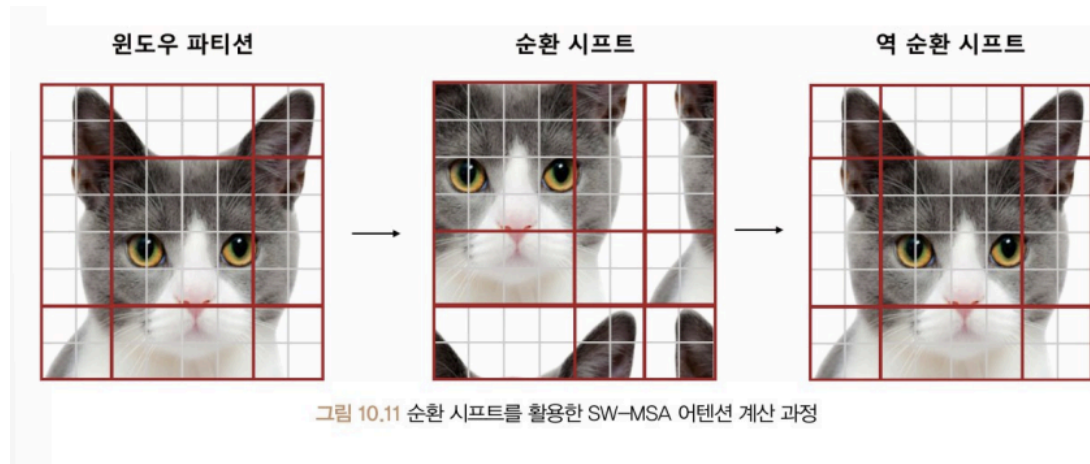


그림 10.10 W-MSA와 SW-MSA 어텐션 영역 비교

- SW-MSA : 윈도우를 가로 또는 세로 방향으로 이동한 후, 인접한 위도 간의 셀프 어텐션을 계산하는 방식(윈도우 사이의 연결성 구축)



- 더 많은 셀프 어텐션 연산을 요구하는 문제를 해결하기 위해 Cyclic Shift와 Attention Mask를 사용
- Cyclic Shift

- 로컬 윈도가 이동할 때 이동한 위치의 정보를 이전 위치에서 가져오는 것을 말하며, 로컬 윈도 사이트 M 보다 작은 $M/2$ 만큼 로컬 윈도들을 이동
- Attention Mask
 - 이동된 위치에서 연산을 수행하지 않도록 방지
- 순환 시프트 단계에서 셀프 어텐션을 수행한 다음 Reverse Cyclic Shift를 사용해 원래 로컬 윈도 구조로 복원

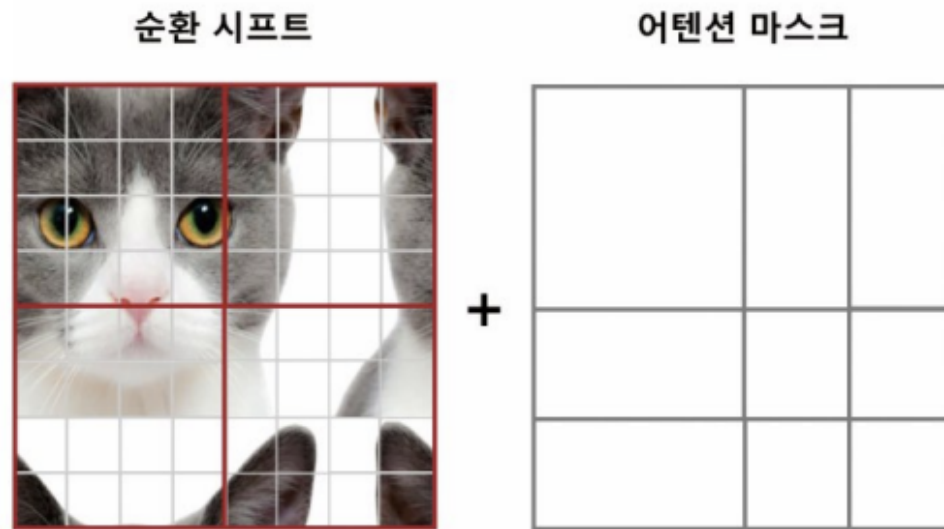


그림 10.12 순환 시프트 단계에서 마스크를 활용한 셀프 어텐션 구조

- ex) 총 9개의 윈도 파티션에 대해 9번의 셀프 어텐션 값을 구해야 했지만 → 4개의 윈도 파티션에 대한 4개의 셀프 어텐션 값만으로도 모든 로컬 윈도 간의 셀프 어텐션 값을 구할 수 있음
- 이동된 윈도에 대한 정보를 보존하면서도, 불필요한 연산을 줄이고 효율적인 계산을 가능하게 한다는 장점



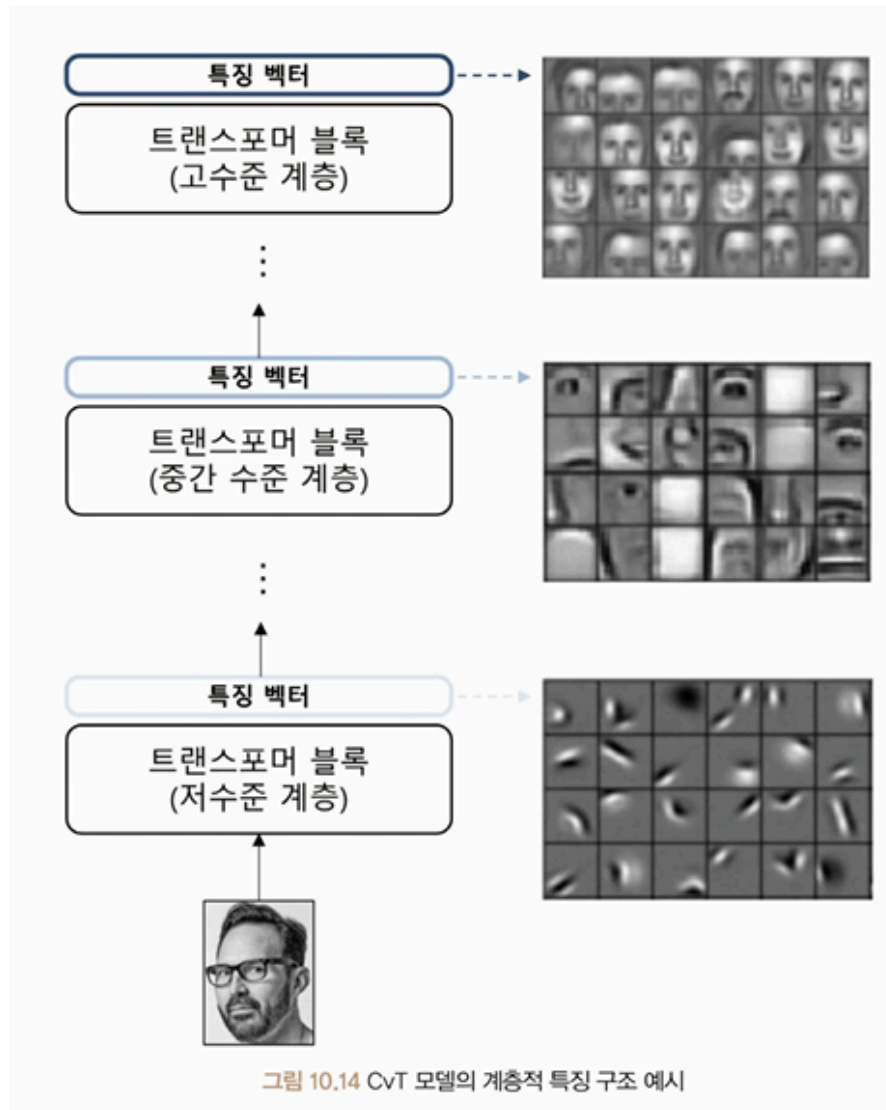
그림 10.13 상대적 위치 편향 계산 과정

- 기존 ViT와 다른 점 ⇒ Relative Position Bais를 고려
 - 로컬 윈도우 안에 있는 패치 간의 상대적인 거리를 임베딩하는 목적을 가짐
 - 패치 간의 상대적인 거리는 X축과 Y축 두 가지 방식으로 표기
 - X축방식: 두 패치 간의 가로 방향 거리를 의미
 - 같은 x축 ⇒ 상대적 거리 0, 아래 ⇒ 1, 위 ⇒ -1
 - Y축 방식: 세로 방향 거리를 의미
 - 같은 y축 ⇒ 상대적 거리 0, 오른쪽 ⇒ -1, 왼쪽 ⇒ 1

CvT

CvT(Convolutional Vision Transformer): 합성곱 신경망 구조에서 활용하는 계층형 구조를 ViT에 적용한 모델

- 합성곱 신경망에서 사용하는 계층적 구조를 ViT에 적용해 이미지의 지역 특징과 전역 특징을 모두 활용해 비교적 적은 수의 매개변수로 높은 성능을 보임



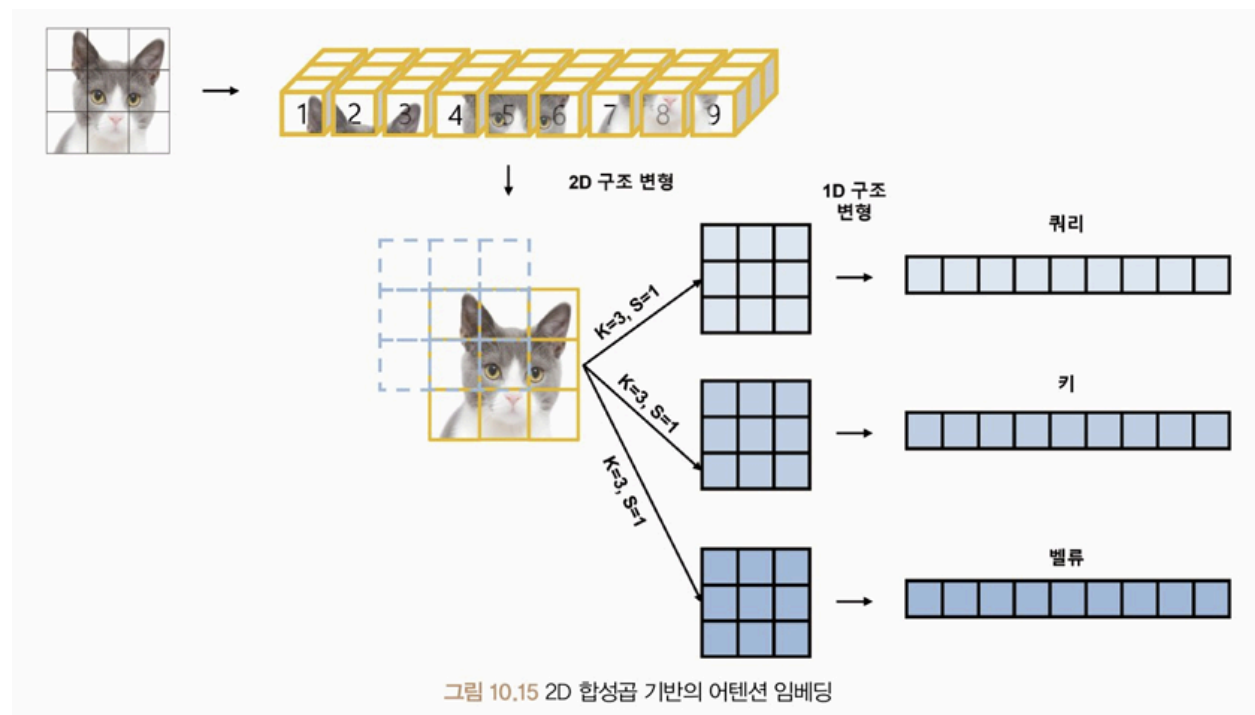
- 얼굴 이미지
 - 저수준 계층에서 선 또는 점에 대한 특징 표현
 - 중간 수준에서는 눈이나 귀와 같은 특징 표현
 - 고수준 계층에서는 얼굴 구조에 대한 특징 표현
- CvT는 이미지를 처리하기 전에 합성곱 연산을 통해 특징 맵을 추출
 - 유의미한 패턴을 추출하고 일정한 크기의 패치로 나눠 트랜스포머 모델의 입력으로 사용

표 10.4 ViT와 CvT 모델 비교

모델	위치 임베딩	패치 임베딩	어텐션 임베딩	계층형 구조
ViT	O	겹치지 않는 선형 임베딩	선형 임베딩	X
CvT	X	겹치는 합성곱 임베딩 (overlapping)	합성곱 임베딩	O

합성곱 토큰 임베딩

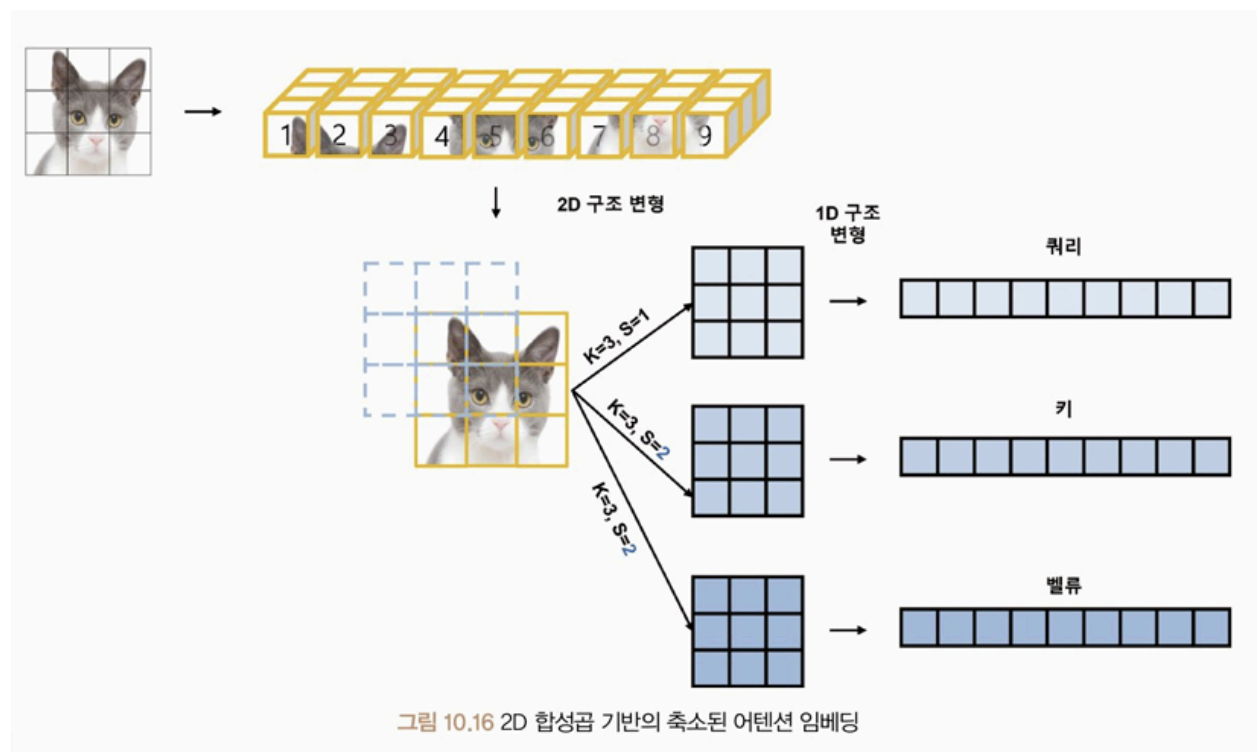
: 2D로 재구성된 토큰 맵에서 중첩 합성곱 연산을 수행하는 임베딩



- ViT에 계층적인 합성곱 신경망의 성질을 추가하기 위해 사용
- 이미지의 2D 구조 보존하면서도 셀프 어텐션을 계산
 - 이미지 패치를 2D 합성곱 임베딩을 사용해 Q,K,V 임베딩으로 변환
 - ex) 10.15같이 3*3 커널 크기(K), 1 간격(S)의 커널을 적용⇒ 패치 크기는 3*3
 - 각각의 패치를 Q,K,V 값에 대해 합성곱 연산을 적용하고 1차원 구조로 재배열에 기존 셀프 어텐션 방법으로 학습하는 방식으로 진행됨

어텐션에 대한 합성곱 임베딩

:2D로 재구성된 토큰맵에서 분리 가능한 합성곱 연산을 적용해 성능 저하 및 계산 복잡도를 낮추는 연산



- 매개변수의 수를 줄이기 위해 Q,K,V 값의 차원 축소
- 그림에서는 간격 2를 적용해 4개의 이미지 패치 생성
- 길이가 다른 쿼리(9), 키(4), 값(4)를 사용해도 기존 셀프 어텐션의 결과값 사이즈가 9*1로 동일하게 출력 가능

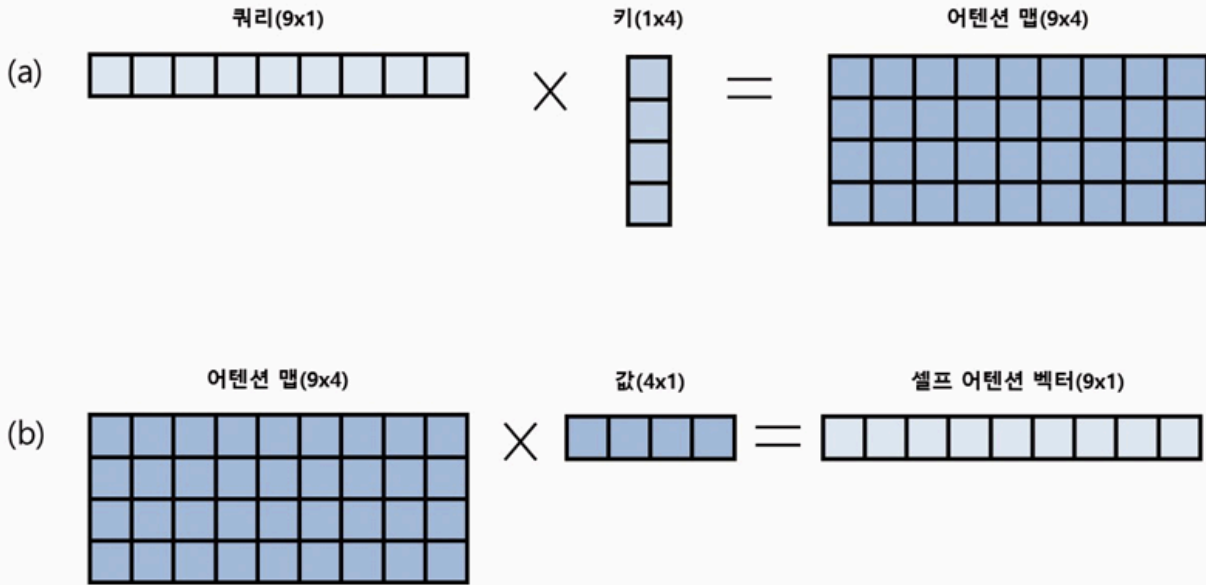


그림 10.17 축소된 셀프 어텐션 임베딩(열 × 행) 예시

- (a)에서는 9개의 쿼리 패치 임베딩과 4개의 키 패치 임베딩에 대한 중요도를 의미하는 Attention Map을 생성
- Attention Map이 생성되면 (b)와 같이 값 패치 임베딩으로 쿼리 어텐션 맵에 대한 가중치를 반영
- Self attention vector $\Rightarrow$ 입력 벡터와 출력 벡터의 패치 길이가 동일 $\Rightarrow$ 계산해야하는 이미지 패치 개수가 줄어들어 연산량 감소
- CvT 모델은 트랜스포머 계층이 깊어질수록 패치 간의 관계를 학습하는 길이를 축소시키는 대신에 벡터의 차원을 증가시켜 모델의 매개변수 수를 대폭 감소시킴
 - ex) 첫번째 트랜스포머 블록의 이미지 패치 크기가 64*64, 임베딩 차원이 128이고 마지막 트랜스포머 블록의 이미지 패치 크기를 4배 감소시켜 8*8로 설정 $\Rightarrow$ 기존 임베딩 차원도 4배 증가시켜 512차원으로 증가

Supporting files